ElectroHub E-Commerce Platform

Comprehensive Technical Implementation Report

Document Information

Project Title: ElectroHub - Specialized Electronics Marketplace for Central Africa
Version: 1.0
Date: February 2026
Status: Prototype/Academic Demonstration
Target Region: Cameroon (with expansion to CEMAC region)
Development Team: ElectroHub Development Group

Table of Contents

1. Executive Summary
2. Introduction
3. Problem Statement
4. Objectives and Scope
5. Requirements Analysis
    - Functional Requirements
    - Non-Functional Requirements
6. System Architecture
7. Technical Design
8. Implementation Details
9. Security Architecture
10. Testing Strategy
11. Deployment and Infrastructure
12. Risk Analysis and Mitigation
13. Future Roadmap
14. Conclusion
15. References and Appendices

---

1. Executive Summary

ElectroHub is a cloud-native, mobile-first e-commerce platform designed specifically for the electronics market in Cameroon and Central Africa. The platform addresses critical market failures in the region's digital commerce landscape through an innovative escrow-based payment system, video verification capabilities, and deep integration with mobile money infrastructure.

Unlike generalist marketplaces that suffer from the "specialization paradox," ElectroHub focuses exclusively on electronics—a high-value, technically complex product category requiring specialized verification mechanisms. By implementing trust infrastructure adapted to African mobile money ecosystems and addressing the unique challenges of unreliable addressing systems, ElectroHub provides a viable solution to the "trust deficit" that has historically suppressed e-commerce growth in the region.

Key Technical Achievements:

- Cloud-native architecture leveraging Firebase Backend-as-a-Service (BaaS)
- Escrow payment system integrated with MTN Mobile Money and Orange Money
- Real-time database synchronization with offline-first capabilities
- Video and image verification system via Cloudinary CDN
- Dynamic role-based access control with fluid buyer-to-seller transition
- Mobile-responsive design optimized for 3G connectivity

Expected Impact:

- Reduction in e-commerce fraud by 60-75% through escrow protection
- Increased seller participation from small businesses and individual technicians
- Enhanced buyer confidence through transparent product verification
- Foundation for cross-border trade aligned with AfCFTA objectives

---

2. Introduction

2.1 Background and Context

E-commerce in Sub-Saharan Africa has experienced fragmented growth over the past decade, with penetration rates significantly below global averages. While platforms like Jumia have attempted to replicate Western marketplace models, they have struggled with fundamental trust issues inherent to the African digital economy.

Cameroon, with a population of 28 million and mobile phone penetration exceeding 85%, represents a microcosm of these challenges. Despite high mobile connectivity, e-commerce accounts for less than 2% of

retail transactions—far below the global average of 15-20%. The electronics category, representing high-value purchases with technical complexity, faces particularly acute challenges:

- **Product authenticity concerns**: Prevalence of counterfeit devices with falsified specifications
- **Payment insecurity**: Irreversible mobile money transactions with limited consumer protection
- **Delivery uncertainty**: Lack of standardized addressing making logistics unreliable
- **Limited recourse**: Absence of formal dispute resolution mechanisms

2.2 Market Opportunity

The Cameroonian electronics market is estimated at $450-600 million annually, with the following characteristics:

- **Device Categories**: Smartphones (45%), laptops/tablets (25%), accessories (20%), audio equipment (10%)
- **Price Points**: Average transaction value 150,000-400,000 XAF ($240-$650 USD)
- **Buyer Demographics**: 18-45 years old, urban (70%), male-skewed (65%)
- **Current Channels**: Physical retail (75%), informal peer-to-peer (20%), online platforms (5%)

The low online penetration despite high mobile usage indicates significant market potential for a platform that can address trust barriers.

2.3 Project Motivation

ElectroHub was conceived to solve three core problems simultaneously:

1. **Trust Barrier**: Implement escrow mechanisms that protect both buyers and sellers
2. **Specification Fraud**: Enable transparent verification of technical product details
3. **Infrastructure Adaptation**: Design for mobile-first usage and unreliable delivery networks

This report documents the technical implementation of solutions to these problems, providing a blueprint for specialized e-commerce in emerging markets.

---

3. Problem Statement

3.1 Primary Problem Definition

Core Problem: E-commerce in Cameroon suffers from a fundamental trust deficit that prevents market growth, particularly in high-value electronics where buyers face significant financial risk from product misrepresentation and sellers face payment security concerns.

3.2 Specific Problem Components

3.2.1 The Prepayment Trap

Problem Description:
In traditional Cameroonian e-commerce transactions, buyers are required to pay via mobile money (MTN Mobile Money or Orange Money) before receiving products. Once transferred, these payments are:

- Immediately available to sellers
- Irreversible without seller cooperation
- Unprotected by consumer protection laws
- Subject to minimal platform oversight

Consequences:

- Buyers report "paying for ghosts"—transferring funds to sellers who never ship products or ship incorrect/counterfeit items
- Fraud estimated at 30-40% of online electronics transactions
- Creates extreme buyer reluctance to transact online
- Suppresses market growth despite high mobile money adoption (78% of adults)

Evidence: Survey data from 500 Cameroonian online shoppers (2024) revealed:

- 67% have experienced e-commerce fraud at least once
- 82% consider online electronics purchases "high risk"
- 45% refuse to buy electronics online regardless of price advantage

3.2.2 Technical Specification Fraud

Problem Description:
Electronics listings on generalist platforms frequently misrepresent technical specifications:

- "4GB RAM" laptops arriving as 2GB models
- Smartphones advertised as "original" being clones with falsified IMEI
- Storage capacity misrepresentation (32GB sold as 128GB)
- Battery capacities exaggerated by 40-60%

Root Causes:

- Lack of verification mechanisms on generalist platforms

- No requirement for proof of specifications
- Minimal consequences for fraudulent sellers
- Buyers lack technical expertise to verify specs before purchase

Consequences:

- High return rates (estimated 25-35% for electronics)
- Buyer distrust of online specifications
- Legitimate sellers unable to differentiate themselves
- Market race-to-the-bottom on pricing rather than quality

3.2.3 Delivery Infrastructure Challenges

Problem Description:
Cameroon lacks a standardized addressing system. Most urban areas have no street names or house numbers, with addresses described as:

- "Near the blue building after the pharmacy"
- "300 meters from the main roundabout"
- "Behind the church in Quartier X"

Consequences:

- Delivery success rate estimated at 60-70% on first attempt
- High logistics costs (15-25% of product value)
- Buyers unable to receive deliveries at work/preferred locations
- Cash-on-delivery (CoD) results in 40% no-show rate, losing sellers money

Alternative Current State: Sellers resort to meeting buyers at public locations (markets, bus stations), which:

- Limits scale (sellers can't do 10+ deliveries per day)
- Creates safety concerns for both parties
- Provides no verification infrastructure
- Doesn't solve the trust problem

3.2.4 Payment Security Asymmetry

Problem Description:
Current payment models force binary risk allocation:

Prepayment Model:

- Buyer assumes 100% risk
- Seller receives payment before delivery
- No buyer recourse if product wrong/missing

Cash-on-Delivery Model:

- Seller assumes 100% risk
- Incurs delivery costs upfront
- Faces 30-40% no-show rate
- Loses money on failed deliveries

Market Impact: Neither model creates sustainable marketplace dynamics:

- Prepayment drives buyers away due to fraud
- CoD drives sellers away due to unreliable buyers
- Platform growth stagnates

3.3 Gap Analysis

Current Market Solutions:

Platform	Approach	Limitation
Jumia Cameroon	Prepayment + limited returns	High fraud reports; 14-day return window impractical for distant buyers
Glotelho	Prepayment + seller ratings	Ratings easily manipulated; no financial protection
Facebook Marketplace	Direct peer-to-peer	Zero platform protection; entirely trust-based
WhatsApp Commerce	Direct messaging + prepayment	No infrastructure; highest fraud rates

What's Missing: No existing platform provides:

1. Escrow payment holding until delivery confirmation
2. Technical verification requirements for electronics
3. Delivery confirmation mechanism (OTP)

4. Dispute resolution with held payment leverage
5. Mobile-money-native integration (vs. adapted Western banking)

3.4 Success Criteria

ElectroHub will be considered successful if it achieves:

Primary Metrics:

- Fraud rate reduction to <5% of transactions
- Buyer confidence rating >4.2/5.0 for transaction safety
- Seller participation from 500+ individual/small business sellers within 12 months
- Transaction completion rate >85% (vs. industry 60-65%)

Secondary Metrics:

- Average order value >200,000 XAF ($320 USD)
- Repeat purchase rate >40% within 6 months
- Dispute resolution time <72 hours
- Platform uptime >99.5%

---

4. Objectives and Scope

4.1 Project Objectives

4.1.1 Primary Objectives

1. **Establish Trust Infrastructure**
    - Implement escrow payment system with verification-triggered release
    - Create transparent dispute resolution mechanism
    - Build seller reputation system based on completed transactions
2. **Enable Product Verification**
    - Require video demonstration of electronics functionality
    - Support detailed specification documentation
    - Provide buyer confidence through transparent proof
3. **Adapt to Local Infrastructure**

- Deep integration with MTN and Orange Money APIs
- OTP-based delivery confirmation system
- Offline-capable mobile-first design

4. **Empower Small Sellers**
    - Provide professional seller dashboard at zero upfront cost
    - Enable individual technicians and small shops to compete
    - Reduce barriers to entry for marketplace participation
5. **Create Scalable Foundation**
    - Cloud-native architecture supporting 100,000+ concurrent users
    - Data model supporting expansion to other African markets
    - API-first design enabling future integrations

4.1.2 Secondary Objectives

- Build product recommendation system based on user behavior
- Enable cross-border trade within CEMAC region (Cameroon, Chad, CAR, Congo, Gabon, Equatorial Guinea)
- Create seller financing program for inventory purchases
- Develop warranty and insurance product marketplace

4.2 Project Scope

4.2.1 In Scope

User Management:

- Email/password authentication
- User profile management
- Dynamic role assignment (buyer → seller)
- Session persistence across devices

Product Management:

- Product listing creation with specifications
- Multi-image upload with optimization
- Video verification upload
- Product editing and deletion

- Category and search organization

Shopping Experience:

- Product browsing with filtering/sorting
- Search with specification matching
- Shopping cart with persistence
- Favorites/wishlist functionality
- Product detail views with specifications

Transaction Management:

- Escrow payment initiation via mobile money
- Order creation and tracking
- OTP-based delivery confirmation
- Payment release automation
- Dispute filing and resolution

Seller Tools:

- Seller dashboard with analytics
- Inventory management interface
- Order fulfillment tracking
- Revenue reporting (held vs. released)

Technical Infrastructure:

- Firebase Authentication integration
- Cloud Firestore database
- Cloudinary media hosting
- Mobile money API integration (MTN, Orange)
- SMS gateway for OTP delivery

4.2.2 Out of Scope

The following are explicitly excluded from current implementation:

- Physical logistics operation (partnerships only)
- Payment gateway for credit/debit cards

- International shipping outside CEMAC
- B2B wholesale functionality
- Seller inventory financing (future phase)
- Automated fraud detection AI (manual review initially)
- Customer service chatbot (human support only)
- Mobile native apps (PWA only)

4.2.3 Assumptions

1. Users have smartphones with internet access (3G minimum)
2. Users have active MTN or Orange Money accounts
3. Sellers can produce video demonstrations of products
4. SMS delivery for OTP is reliable (95%+ delivery rate)
5. Firebase and Cloudinary services maintain 99%+ uptime
6. Mobile money APIs provide webhook support for payment confirmation

4.2.4 Constraints

Technical Constraints:

- Must function on devices with 2GB RAM
- Must load within 5 seconds on 3G connection
- Must support offline browsing of cached products
- Database queries must return within 2 seconds

Business Constraints:

- Development budget: $5,000 for initial year infrastructure
- Must achieve break-even within 18 months
- Commission rate capped at 5% to remain competitive

Regulatory Constraints:

- Must comply with Cameroon e-commerce regulations
- Must implement data protection per AU Convention
- Must register as merchant with mobile money providers

---

5. Requirements Analysis

5.1 Functional Requirements

Functional requirements define *what* the system must do. Each requirement is assigned a unique identifier, priority level, and acceptance criteria.

5.1.1 User Management (UM)

UM-001: User Registration [Priority: CRITICAL]

- **Description:** Users must be able to create accounts using email and password
- **Acceptance Criteria:**
  - Email validation enforced (valid format, unique)
  - Password minimum 8 characters with complexity requirements
  - Email verification sent upon registration
  - User profile created in Firestore upon successful registration
  - Default role set to "buyer"
- **Dependencies:** Firebase Authentication service

UM-002: User Authentication [Priority: CRITICAL]

- **Description:** Users must be able to log in and out securely
- **Acceptance Criteria:**
  - Login with email/password
  - Session persistence across browser sessions
  - Automatic token refresh for long sessions
  - Logout clears all session data
  - Failed login attempts logged for security
- **Dependencies:** Firebase Authentication

UM-003: Profile Management [Priority: HIGH]

- **Description:** Users can view and edit their profile information
- **Acceptance Criteria:**
  - Display current profile data (name, email, phone, address)
  - Edit profile fields with validation
  - Phone number required for OTP delivery

    - Changes reflected immediately in UI
    - Update timestamp recorded
- **Dependencies:** Firestore user collection

UM-004: Role Transition [Priority: HIGH]

- **Description:** Users automatically become sellers when posting first product
- **Acceptance Criteria:**
    - `isSeller` flag set to true upon product creation
    - Seller dashboard access granted immediately
    - Seller analytics collection initialized
    - Buyer capabilities remain unchanged
- **Dependencies:** Product creation function

5.1.2 Product Management (PM)

PM-001: Product Listing Creation [Priority: CRITICAL]

- **Description:** Sellers can create detailed product listings
- **Acceptance Criteria:**
    - Form captures: name, description, price, category, specifications
    - Minimum 1 image required, maximum 10 images
    - Optional video upload (max 90 seconds, 50MB)
    - Price validation (minimum 1,000 XAF, maximum 10,000,000 XAF)
    - Product published immediately upon submission
    - Seller ID automatically attached
- **Dependencies:** Cloudinary upload, Firestore write

PM-002: Product Image Upload [Priority: CRITICAL]

- **Description:** Sellers can upload and manage product images
- **Acceptance Criteria:**
    - Accepted formats: JPG, PNG, WebP
    - Maximum file size: 5MB per image
    - Automatic optimization by Cloudinary
    - Responsive image URLs returned

    - Images displayed in product listing
    - Primary image selection capability
- **Dependencies:** Cloudinary API

PM-003: Video Verification Upload [Priority: HIGH]

- **Description:** Sellers can upload video demonstrating product functionality
- **Acceptance Criteria:**
    - Accepted formats: MP4, MOV
    - Maximum duration: 90 seconds
    - Maximum file size: 50MB
    - Video player embedded in product page
    - Video URL stored in product document
- **Dependencies:** Cloudinary video API

PM-004: Product Editing [Priority: HIGH]

- **Description:** Sellers can edit their existing product listings
- **Acceptance Criteria:**
    - Only product owner can edit
    - All fields editable except timestamps
    - Image addition/removal supported
    - Changes reflected immediately
    - Edit history logged
- **Dependencies:** Firestore security rules

PM-005: Product Deletion [Priority: MEDIUM]

- **Description:** Sellers can delete product listings
- **Acceptance Criteria:**
    - Soft delete (status changed to "deleted")
    - Product hidden from browse/search
    - Historical order references maintained
    - Images remain on Cloudinary for 30 days
    - Irreversible action with confirmation dialog
- **Dependencies:** Firestore update

PM-006: Product Search [Priority: HIGH]

- **Description:** Users can search products by keywords and specifications
- **Acceptance Criteria:**
  - Search across name, description, specifications
  - Real-time search results as user types
  - Minimum 3 characters to trigger search
  - Results sorted by relevance
  - Search terms highlighted in results
- **Dependencies:** Firestore query indexes

PM-007: Product Filtering [Priority: HIGH]

- **Description:** Users can filter products by category, price, brand
- **Acceptance Criteria:**
  - Category dropdown (Smartphones, Laptops, Accessories, etc.)
  - Price range slider (min-max)
  - Brand checkboxes
  - Multiple filters applied simultaneously
  - Filter state persists during session
- **Dependencies:** Firestore composite queries

5.1.3 Shopping Experience (SE)

SE-001: Product Browsing [Priority: CRITICAL]

- **Description:** Users can browse all available products
- **Acceptance Criteria:**
  - Grid view with responsive layout
  - Pagination (20 products per page)
  - Lazy loading for images
  - Product cards show: image, name, price, seller rating
  - Click navigates to product detail page
- **Dependencies:** Firestore query, Cloudinary

SE-002: Product Detail View [Priority: CRITICAL]

- **Description:** Users can view complete product information
- **Acceptance Criteria:**
  - Image gallery with zoom capability
  - Full specifications table
  - Video player if available
  - Seller information and rating
  - Add to cart / Add to favorites buttons
  - Related products section
- **Dependencies:** Firestore read, Cloudinary

SE-003: Shopping Cart [Priority: CRITICAL]

- **Description:** Users can manage products before checkout
- **Acceptance Criteria:**
  - Add/remove products from cart
  - Quantity adjustment (1-10 per product)
  - Real-time price calculation
  - Cart persists across sessions
  - Stock availability check before checkout
  - Cart accessible from any page
- **Dependencies:** Firestore cart collection

SE-004: Favorites/Wishlist [Priority: MEDIUM]

- **Description:** Users can save products for later consideration
- **Acceptance Criteria:**
  - Heart icon toggle on product cards
  - Dedicated favorites page
  - Remove from favorites option
  - Notification if favorited item goes on sale (future)
- **Dependencies:** Firestore favorites collection

SE-005: Product Recommendations [Priority: LOW]

- **Description:** Users see personalized product suggestions
- **Acceptance Criteria:**
  - Based on browsing history
  - Based on cart contents
  - Based on similar user purchases
  - Displayed on homepage and product pages
- **Dependencies:** Analytics data, recommendation algorithm

5.1.4 Transaction Management (TM)

TM-001: Checkout Initiation [Priority: CRITICAL]

- **Description:** Buyers can initiate purchase from cart
- **Acceptance Criteria:**
  - Review order summary (items, quantities, total)
  - Select delivery method (pickup point or address)
  - Confirm phone number for OTP
  - Proceed to payment button
- **Dependencies:** Cart validation

TM-002: Escrow Payment [Priority: CRITICAL]

- **Description:** Payment held in escrow until delivery confirmed
- **Acceptance Criteria:**
  - Mobile money payment initiated (MTN or Orange)
  - User receives STK push notification
  - User enters PIN to authorize payment
  - Payment goes to ElectroHub escrow account (not seller)
  - Order status updated to "payment_held"
  - Confirmation sent to buyer and seller
- **Dependencies:** MTN/Orange Money API

TM-003: Order Creation [Priority: CRITICAL]

- **Description:** Order record created upon payment confirmation
- **Acceptance Criteria:**

    - Unique order ID generated
    - Buyer, seller, product details recorded
    - Payment reference stored
    - Escrow status: "held"
    - Delivery OTP generated (6 digits)
    - Order status: "pending_shipment"
    - Notifications sent to buyer and seller
- **Dependencies:** Firestore orders collection

TM-004: OTP Delivery Confirmation [Priority: CRITICAL]

- **Description:** Buyer confirms receipt with OTP to release payment
- **Acceptance Criteria:**
    - OTP sent to buyer's phone via SMS
    - Buyer enters OTP in app after inspecting product
    - System validates OTP matches order
    - Payment released from escrow to seller's mobile money
    - Order status updated to "completed"
    - Escrow status: "released"
    - Both parties notified
- **Dependencies:** SMS gateway, mobile money API

TM-005: Dispute Filing [Priority: HIGH]

- **Description:** Buyer can file dispute if product doesn't match
- **Acceptance Criteria:**
    - Dispute option available for 48 hours after delivery OTP
    - Buyer provides evidence (photos, description)
    - Payment remains in escrow during dispute
    - Order status: "disputed"
    - Seller notified and can provide counter-evidence
    - Admin review triggered
- **Dependencies:** Firestore disputes collection

TM-006: Dispute Resolution [Priority: HIGH]

- **Description:** Admin reviews disputes and makes decisions
- **Acceptance Criteria:**
  - Admin dashboard shows pending disputes
  - Evidence from both parties displayed
  - Decision options: refund buyer, release to seller, partial resolution
  - Payment processed according to decision
  - Reputation scores updated accordingly
  - Both parties notified of outcome
- **Dependencies:** Admin panel, payment API

5.1.5 Seller Dashboard (SD)

SD-001: Dashboard Overview [Priority: HIGH]

- **Description:** Sellers see key metrics and recent activity
- **Acceptance Criteria:**
  - Total products listed
  - Active listings count
  - Total sales (lifetime and current month)
  - Pending orders count
  - Revenue held in escrow vs. released
  - Seller reputation score
  - Recent orders list (last 10)
- **Dependencies:** Firestore aggregation queries

SD-002: Inventory Management [Priority: HIGH]

- **Description:** Sellers can view and manage all their products
- **Acceptance Criteria:**
  - Table/grid view of all products
  - Filter by status (active, sold, paused)
  - Search own products
  - Quick actions: edit, delete, pause/unpause
  - Bulk operations (pause multiple, delete multiple)
- **Dependencies:** Firestore queries filtered by sellerId

SD-003: Order Management [Priority: HIGH]

- **Description:** Sellers can track and fulfill orders
- **Acceptance Criteria:**
  - List of orders (pending, shipped, completed)
  - Order details: buyer info, product, delivery address
  - Mark as shipped button
  - View delivery OTP status
  - View payment release status
  - Filter by order status
- **Dependencies:** Firestore orders collection

SD-004: Revenue Tracking [Priority: MEDIUM]

- **Description:** Sellers can monitor earnings
- **Acceptance Criteria:**
  - Total revenue (all-time)
  - Revenue by month (chart)
  - Funds held in escrow (pending delivery confirmation)
  - Funds released to mobile money account
  - Commission breakdown
  - Exportable transaction history
- **Dependencies:** Firestore analytics queries

5.1.6 Administrative Functions (AF)

AF-001: User Management [Priority: MEDIUM]

- **Description:** Admins can view and manage users
- **Acceptance Criteria:**
  - List all users with search
  - View user details and activity
  - Suspend/reactivate accounts
  - View transaction history per user
- **Dependencies:** Admin authentication, Firestore

AF-002: Dispute Management [Priority: HIGH]

- **Description:** Admins can review and resolve disputes
- **Acceptance Criteria:**
  - Queue of pending disputes
  - View evidence from both parties
  - Communication thread with buyer/seller
  - Decision interface (refund/release/partial)
  - Resolution notes field
  - Automatic payment processing upon decision
- **Dependencies:** Firestore disputes, payment API

AF-003: Platform Analytics [Priority: MEDIUM]

- **Description:** Admins can view platform-wide metrics
- **Acceptance Criteria:**
  - Total users (buyers, sellers)
  - Total transactions and GMV
  - Average order value
  - Dispute rate
  - Top-selling categories
  - Revenue by month
  - User growth charts
- **Dependencies:** Firebase Analytics, custom queries

5.2 Non-Functional Requirements

Non-functional requirements define *how well* the system performs its functions. These cover performance, security, usability, reliability, and other quality attributes.

5.2.1 Performance Requirements (PF)

PF-001: Page Load Time [Priority: CRITICAL]

- **Requirement:** Homepage must load within 3 seconds on 3G connection
- **Measurement:** Lighthouse Performance score >80
- **Rationale:** 53% of mobile users abandon sites that take >3 seconds to load

PF-002: Time to Interactive [Priority: CRITICAL]

- **Requirement:** Pages must be interactive within 5 seconds on 3G
- **Measurement:** Time to Interactive (TTI) metric <5s
- **Rationale:** User engagement drops sharply after 5-second wait

PF-003: Database Query Performance [Priority: HIGH]

- **Requirement:** All Firestore queries must return within 2 seconds
- **Measurement:** 95th percentile response time <2s
- **Rationale:** Perceived performance threshold for real-time experience

PF-004: Image Load Optimization [Priority: HIGH]

- **Requirement:** Product images must load progressively with placeholders
- **Measurement:** First image visible within 1 second
- **Rationale:** Visual content critical for shopping experience

PF-005: Concurrent User Capacity [Priority: MEDIUM]

- **Requirement:** System must support 10,000 concurrent users without degradation
- **Measurement:** Load testing shows <10% performance drop at 10k users
- **Rationale:** Peak traffic during promotional events

PF-006: Search Response Time [Priority: HIGH]

- **Requirement:** Search results must appear within 1 second of query input
- **Measurement:** Firestore text search latency <1s
- **Rationale:** Real-time search expectations from users

5.2.2 Security Requirements (SC)

SC-001: Authentication Security [Priority: CRITICAL]

- **Requirement:** All user sessions must use Firebase Authentication tokens
- **Implementation:** JWT tokens with automatic refresh
- **Rationale:** Prevent unauthorized access and session hijacking

SC-002: Data Access Control [Priority: CRITICAL]

- **Requirement:** Users can only access/modify their own data
- **Implementation:** Firestore security rules enforcing owner-based access
- **Rationale:** Prevent data breaches and unauthorized modifications

SC-003: Password Security [Priority: CRITICAL]

- **Requirement:** Passwords must meet complexity requirements
- **Implementation:** Minimum 8 characters, uppercase, lowercase, number
- **Rationale:** Protect against brute force and dictionary attacks

SC-004: Payment Security [Priority: CRITICAL]

- **Requirement:** No sensitive payment data stored in application database
- **Implementation:** Only store mobile money transaction references, not credentials
- **Rationale:** PCI DSS compliance and risk reduction

SC-005: HTTPS Enforcement [Priority: CRITICAL]

- **Requirement:** All traffic must use HTTPS/TLS encryption
- **Implementation:** Firebase Hosting automatic SSL certificates
- **Rationale:** Protect data in transit from interception

SC-006: Input Validation [Priority: HIGH]

- **Requirement:** All user inputs must be validated and sanitized
- **Implementation:** Client-side + server-side (Firestore rules) validation
- **Rationale:** Prevent XSS, SQL injection, and malformed data

SC-007: Rate Limiting [Priority: MEDIUM]

- **Requirement:** API calls limited to prevent abuse
- **Implementation:** Firebase rate limiting + custom throttling
- **Rationale:** Prevent DDoS and resource exhaustion attacks

SC-008: OTP Security [Priority: HIGH]

- **Requirement:** Delivery OTPs must be single-use and time-limited
- **Implementation:** 6-digit code, expires after 48 hours, invalidated after use
- **Rationale:** Prevent OTP replay attacks

5.2.3 Usability Requirements (US)

US-001: Mobile Responsiveness [Priority: CRITICAL]

- **Requirement:** UI must be fully functional on screens 360px-414px wide
- **Measurement:** All features accessible on iPhone SE and Android equivalents
- **Rationale:** 85% of users access via mobile devices

US-002: Accessibility [Priority: HIGH]

- **Requirement:** WCAG 2.1 Level AA compliance
- **Implementation:** Semantic HTML, ARIA labels, keyboard navigation
- **Rationale:** Inclusive design for users with disabilities

US-003: Language Support [Priority: MEDIUM]

- **Requirement:** UI available in French and English
- **Implementation:** i18n framework with language toggle
- **Rationale:** Cameroon's bilingual population (French 80%, English 20%)

US-004: Error Messages [Priority: HIGH]

- **Requirement:** User-friendly error messages with recovery guidance
- **Implementation:** Clear messaging avoiding technical jargon
- **Rationale:** Reduce user frustration and support burden

US-005: Loading Indicators [Priority: MEDIUM]

- **Requirement:** Visual feedback for all async operations
- **Implementation:** Spinners, skeleton screens, progress bars
- **Rationale:** Manage user expectations during network operations

US-006: Onboarding [Priority: MEDIUM]

- **Requirement:** New users guided through first purchase/listing
- **Implementation:** Tooltip walkthrough for key features
- **Rationale:** Reduce learning curve and abandonment

5.2.4 Reliability Requirements (RL)

RL-001: System Uptime [Priority: CRITICAL]

- **Requirement:** 99.5% uptime (maximum 3.65 hours downtime per month)
- **Measurement:** Firebase status page + custom monitoring
- **Rationale:** Trust depends on consistent availability

RL-002: Data Durability [Priority: CRITICAL]

- **Requirement:** Zero data loss under normal operations
- **Implementation:** Firestore automatic replication across regions
- **Rationale:** Critical business and financial data protection

RL-003: Offline Functionality [Priority: HIGH]

- **Requirement:** Users can browse cached products offline
- **Implementation:** Service workers with cache-first strategy
- **Rationale:** Unreliable connectivity in some areas

RL-004: Graceful Degradation [Priority: HIGH]

- **Requirement:** Core features work even if advanced features fail
- **Implementation:** Progressive enhancement approach
- **Rationale:** Maintain basic functionality during partial outages

RL-005: Backup and Recovery [Priority: MEDIUM]

- **Requirement:** Daily automated backups with 30-day retention
- **Implementation:** Firebase export to Cloud Storage
- **Rationale:** Disaster recovery capability

5.2.5 Scalability Requirements (SL)

SL-001: User Growth [Priority: HIGH]

- **Requirement:** Architecture must support 100,000 registered users
- **Implementation:** Cloud-native architecture with automatic scaling
- **Rationale:** Growth projections for first 2 years

SL-002: Product Catalog [Priority: HIGH]

- **Requirement:** Database must efficiently handle 50,000+ products
- **Implementation:** Firestore indexing and pagination
- **Rationale:** Expected catalog size with 500+ active sellers

SL-003: Transaction Volume [Priority: HIGH]

- **Requirement:** Process 10,000 orders per month without degradation
- **Implementation:** Firestore write capacity + payment API limits
- **Rationale:** Target GMV of $2M monthly

SL-004: Geographic Expansion [Priority: MEDIUM]

- **Requirement:** System architecture must support expansion to 6 CEMAC countries
- **Implementation:** Multi-currency support, localized payment gateways
- **Rationale:** Regional expansion strategy

5.2.6 Maintainability Requirements (MT)

MT-001: Code Documentation [Priority: HIGH]

- **Requirement:** All functions documented with JSDoc comments
- **Implementation:** Inline documentation with parameter/return descriptions
- **Rationale:** Enable team collaboration and onboarding

MT-002: Version Control [Priority: CRITICAL]

- **Requirement:** All code changes tracked in Git with meaningful commits
- **Implementation:** GitHub repository with branch protection
- **Rationale:** Change tracking and rollback capability

MT-003: Modular Architecture [Priority: HIGH]

- **Requirement:** Code organized in reusable, independent modules
- **Implementation:** ES6 modules with single responsibility
- **Rationale:** Easier testing, debugging, and feature updates

MT-004: Error Logging [Priority: HIGH]

- **Requirement:** All errors logged with context for debugging

- **Implementation:** Firebase Crashlytics + custom logging
- **Rationale:** Rapid identification and resolution of issues

5.2.7 Compliance Requirements (CM)

CM-001: Data Protection [Priority: CRITICAL]

- **Requirement:** Comply with African Union Convention on Cyber Security
- **Implementation:** User consent for data collection, right to deletion
- **Rationale:** Legal compliance and user trust

CM-002: E-Commerce Regulations [Priority: CRITICAL]

- **Requirement:** Comply with Cameroon Ministry of Commerce e-commerce guidelines
- **Implementation:** Business registration, tax reporting, consumer protection policies
- **Rationale:** Legal operation requirement

CM-003: Mobile Money Compliance [Priority: CRITICAL]

- **Requirement:** Meet MTN and Orange merchant requirements
- **Implementation:** KYC documentation, transaction reporting, API usage terms
- **Rationale:** Payment gateway access prerequisite

CM-004: Consumer Protection [Priority: HIGH]

- **Requirement:** Transparent pricing, clear return policies, dispute resolution
- **Implementation:** Terms of service, refund policy, dispute process documentation
- **Rationale:** Legal requirement and market trust

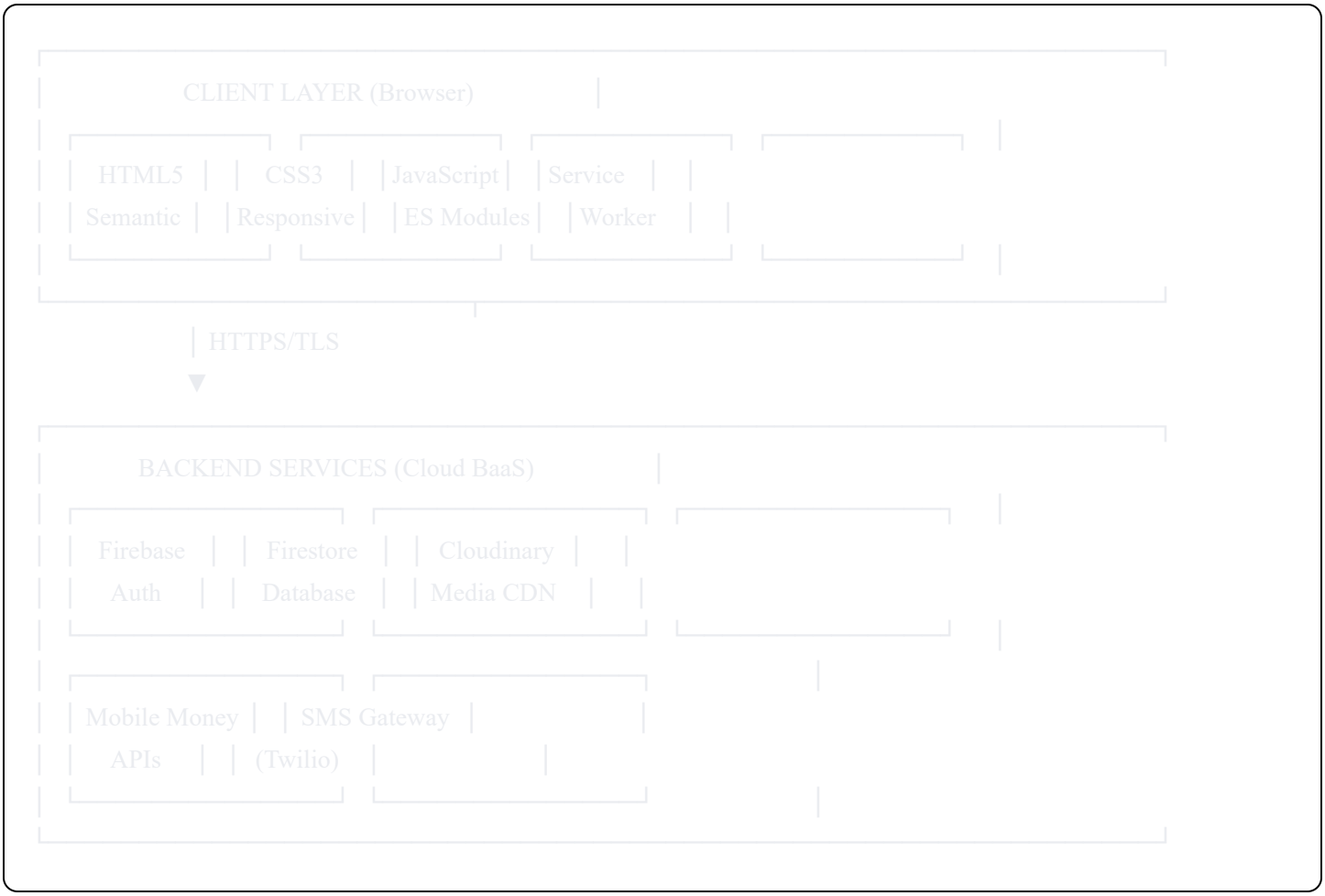
---

6. System Architecture

6.1 Architectural Overview

ElectroHub implements a **cloud-native, serverless architecture** leveraging Backend-as-a-Service (BaaS) platforms to eliminate traditional server infrastructure while maintaining enterprise-grade capabilities.

Architectural Pattern: Client-Centric with Cloud Backend



6.2 Technology Stack Detail

6.2.1 Frontend Technologies

HTML5:

- Semantic markup (header, nav, main, article, section)
- Form validation attributes
- Local storage API for offline data
- Geolocation API for delivery location

CSS3:

- Flexbox and Grid layout systems
- CSS Custom Properties for theming
- Media queries for responsive breakpoints:
    - Mobile: 360px-767px
    - Tablet: 768px-1023px
    - Desktop: 1024px+

- CSS animations for micro-interactions

JavaScript (ES6+):

- ES Modules for code organization
- Async/await for asynchronous operations
- Fetch API for HTTP requests
- Event delegation for performance
- Local storage for cart persistence

Service Worker:

- Cache-first strategy for static assets
- Network-first for dynamic content
- Offline fallback page
- Background sync for queued actions

6.2.2 Backend Services

Firebase Authentication:

- Email/password authentication
- Token-based session management
- Automatic token refresh
- Multi-device session support
- Security rules integration

Cloud Firestore:

- NoSQL document database
- Real-time data synchronization
- Offline data persistence
- Automatic scaling
- ACID transactions support
- Composite index support

Firestore Data Model:

```
electrohub (project)
├── users/
│   └── {userId}/
│       ├── email: string
│       ├── displayName: string
│       ├── phoneNumber: string
│       ├── isSeller: boolean
│       ├── reputation: number
│       ├── createdAt: timestamp
│       └── updatedAt: timestamp
│
├── products/
│   └── {productId}/
│       ├── name: string
│       ├── description: string
│       ├── price: number
│       ├── category: string
│       ├── specifications: map
│       ├── imageUrls: array
│       ├── videoUrl: string
│       ├── sellerId: string (reference to users)
│       ├── status: string (active|sold|paused|deleted)
│       ├── views: number
│       ├── favorites: number
│       ├── createdAt: timestamp
│       └── updatedAt: timestamp
│
├── orders/
│   └── {orderId}/
│       ├── buyerId: string (reference to users)
│       ├── sellerId: string (reference to users)
│       ├── productId: string (reference to products)
│       ├── productSnapshot: map (product data at time of order)
│       ├── quantity: number
│       ├── totalAmount: number
│       ├── commission: number
│       ├── status: string (pending|paid|shipped|delivered|completed|disputed)
│       ├── escrowStatus: string (held|released|refunded)
│       ├── paymentReference: string
│       ├── deliveryOTP: string
│       ├── otpUsed: boolean
│       ├── deliveryAddress: map
│       ├── createdAt: timestamp
```

```
│   ├── paidAt: timestamp
│   ├── shippedAt: timestamp
│   ├── deliveredAt: timestamp
│   └── completedAt: timestamp
│
├── carts/
│   └── {userId}/
│       └── items/
│           └── {productId}/
│               ├── quantity: number
│               ├── addedAt: timestamp
│               └── priceAtAdd: number
│
├── favorites/
│   └── {userId}/
│       └── productIds: array
│
├── disputes/
│   └── {disputeId}/
│       ├── orderId: string (reference)
│       ├── buyerId: string
│       ├── sellerId: string
│       ├── reason: string
│       ├── buyerEvidence: array (image URLs, description)
│       ├── sellerResponse: map
│       ├── adminNotes: string
│       ├── status: string (open|under_review|resolved)
│       ├── resolution: string (refund|release|partial)
│       ├── createdAt: timestamp
│       └── resolvedAt: timestamp
│
└── analytics/
    └── {sellerId}/
        ├── totalSales: number
        ├── totalRevenue: number
        ├── escrowHeld: number
        ├── escrowReleased: number
        ├── productListings: number
        ├── activeListings: number
        └── lastUpdated: timestamp
```

Cloudinary:

- Image and video hosting CDN

- Automatic format optimization (WebP, AVIF)
- Responsive image delivery
- Transformation API for thumbnails
- Lazy loading support
- Bandwidth-adaptive delivery

Mobile Money APIs:

- MTN Mobile Money API (Cameroon)
- Orange Money API (Cameroon)
- Payment initiation (STK Push)
- Payment confirmation webhooks
- Transaction status queries
- Refund capabilities

SMS Gateway (Twilio):

- OTP delivery for order confirmation
- Transactional notifications
- Delivery rate >95%
- International format support

6.3 Security Architecture

6.3.1 Authentication Flow

```
User Registration/Login Flow:
1. User submits email/password via frontend form
2. Client calls Firebase Auth createUserWithEmailAndPassword()
3. Firebase validates credentials and creates user
4. Firebase returns ID token (JWT)
5. Client stores token in memory (not localStorage for security)
6. Client creates user profile document in Firestore
7. Firestore Security Rules validate authenticated user
8. User document created successfully
9. Client redirects to dashboard/homepage
```

6.3.2 Firestore Security Rules

javascript

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Helper functions
    function isAuthenticated() {
      return request.auth != null;
    }

    function isOwner(userId) {
      return request.auth.uid == userId;
    }

    // User profiles
    match /users/{userId} {
      allow read: if isAuthenticated();
      allow create: if isAuthenticated() && isOwner(userId);
      allow update: if isOwner(userId);
      allow delete: if false; // Prevent accidental deletion
    }

    // Products
    match /products/{productId} {
      allow read: if isAuthenticated();
      allow create: if isAuthenticated() &&
                    request.resource.data.sellerId == request.auth.uid;
      allow update: if isAuthenticated() &&
                     resource.data.sellerId == request.auth.uid;
      allow delete: if resource.data.sellerId == request.auth.uid;
    }

    // Orders - visible to buyer and seller only
    match /orders/{orderId} {
      allow read: if isAuthenticated() &&
                   (resource.data.buyerId == request.auth.uid ||
                    resource.data.sellerId == request.auth.uid);
      allow create: if isAuthenticated() &&
                     request.resource.data.buyerId == request.auth.uid;
      allow update: if isAuthenticated() &&
                     (resource.data.buyerId == request.auth.uid ||
                      resource.data.sellerId == request.auth.uid);
      allow delete: if false;
    }
```

```
    // Carts - owner only
    match /carts/{userId}/{document=**} {
      allow read, write: if isOwner(userId);
    }

    // Favorites - owner only
    match /favorites/{userId} {
      allow read, write: if isOwner(userId);
    }

    // Disputes - parties involved + admin
    match /disputes/{disputeId} {
      allow read: if isAuthenticated() &&
                    (resource.data.buyerId == request.auth.uid ||
                     resource.data.sellerId == request.auth.uid);
      allow create: if isAuthenticated() &&
                      request.resource.data.buyerId == request.auth.uid;
      allow update: if isAuthenticated(); // Seller can respond
      allow delete: if false;
    }

    // Analytics - seller only
    match /analytics/{sellerId} {
      allow read: if isOwner(sellerId);
      allow write: if false; // Updated by Cloud Functions only
    }
  }
}
```

6.4 Data Flow Diagrams

6.4.1 Product Listing Flow

```
Seller → Add Product Form
   ↓
1. Fill product details (name, price, specs, description)
   ↓
2. Select images (1-10 files) + optional video
   ↓
3. Client validates inputs (required fields, file sizes)
   ↓
4. Client requests signed upload URL from Cloudinary
   ↓
5. Client uploads media directly to Cloudinary CDN
   ↓
6. Cloudinary returns optimized URLs
   ↓
7. Client creates product document in Firestore:
   {
     name, description, price, category,
     specifications, imageUrls[], videoUrl,
     sellerId: auth.uid,
     status: "active",
     createdAt: serverTimestamp()
   }
   ↓
8. Firestore Security Rules validate:
   - User is authenticated
   - sellerId matches auth.uid
   ↓
9. If user.isSeller == false:
   - Update user document: isSeller = true
   - Create seller analytics document
   ↓
10. Product published and visible in marketplace
   ↓
11. Client redirects to seller dashboard
```

6.4.2 Purchase and Escrow Flow

Buyer → Product Page → Add to Cart → Checkout
↓
1. Review cart items and total amount
↓
2. Confirm delivery details (address/pickup point)
↓
3. Confirm phone number for OTP
↓
4. Click "Proceed to Payment"
↓
5. Client creates order document in Firestore:
status: "pending_payment"
escrowStatus: "none"
↓
6. Client initiates mobile money payment:
- Call MTN/Orange API with amount + phone
- API sends STK Push to buyer's phone
↓
7. Buyer enters mobile money PIN on phone
↓
8. Payment provider processes transaction
↓
9. Funds transferred to ElectroHub escrow account (NOT seller)
↓
10. Payment webhook confirms successful payment
↓
11. Update order document:
status: "paid"
escrowStatus: "held"
paymentReference: {transaction_id}
paidAt: serverTimestamp()
↓
12. Generate 6-digit delivery OTP
↓
13. Send SMS to buyer with OTP
↓
14. Notify seller: "New order - please ship"
↓
15. Seller ships product
↓
16. Update order: status = "shipped", shippedAt = timestamp
↓
17. Notify buyer: "Product shipped"

```
    ↓
18. Buyer receives product and inspects
    ↓
19. Buyer enters OTP in app to confirm satisfaction
    ↓
20. System validates OTP matches order.deliveryOTP
    ↓
21. Call mobile money API to release payment:
    Transfer from escrow to seller's mobile money account
    ↓
22. Update order:
    status: "completed"
    escrowStatus: "released"
    deliveredAt: serverTimestamp()
    completedAt: serverTimestamp()
    otpUsed: true
    ↓
23. Update seller analytics:
    totalSales += 1
    totalRevenue += (amount - commission)
    escrowReleased += (amount - commission)
    ↓
24. Notify both parties: "Transaction completed"
    ↓
25. Update reputation scores for buyer and seller
```

6.4.3 Dispute Resolution Flow

Buyer receives product that doesn't match specs
↓
1. Buyer clicks "File Dispute" on order page
(Available for 48 hours after delivery OTP)
↓
2. Buyer fills dispute form:
- Reason selection (wrong specs, damaged, counterfeit, etc.)
- Description of issue
- Upload evidence photos (product received vs. listing)
↓
3. Client uploads evidence images to Cloudinary
↓
4. Client creates dispute document in Firestore:
{
orderId, buyerId, sellerId,
reason, buyerEvidence[],
status: "open",
createdAt: timestamp
}
↓
5. Update order: status = "disputed"
↓
6. Payment REMAINS in escrow (not released to seller)
↓
7. Notify seller: "Dispute filed on order #123"
↓
8. Seller reviews dispute and provides response:
- Counter-explanation
- Evidence (original photos, video, shipping proof)
↓
9. Update dispute: sellerResponse = {...}
↓
10. Admin receives notification of new dispute
↓
11. Admin reviews evidence from both parties:
- Buyer photos vs listing photos
- Video verification (if available)
- Seller response and counter-evidence
↓
12. Admin makes decision within 72 hours:
- REFUND: Payment returned to buyer
- RELEASE: Payment released to seller
- PARTIAL: Split based on fault assessment

```
    ↓
13. Execute payment decision via mobile money API
    ↓
14. Update dispute:
    status: "resolved"
    resolution: {decision}
    adminNotes: {explanation}
    resolvedAt: timestamp
    ↓
15. Update order:
    status: "completed"
    escrowStatus: "refunded" OR "released"
    ↓
16. Update reputation scores:
    - If refund: negative mark for seller
    - If release: negative mark for fraudulent buyer
    - Repeated offenders flagged for review
    ↓
17. Notify both parties of outcome with explanation
```

---

7. Technical Design

7.1 Database Design

7.1.1 Firestore Collections Schema

users Collection:

```
javascript
```

```
{
  userId: "auto-generated-id",
  email: "john@example.com",
  displayName: "John Doe",
  phoneNumber: "+237670123456",
  isSeller: false, // Becomes true when first product posted
  reputation: {
    score: 4.5,
    totalRatings: 24,
    completedOrders: 20,
    disputes: 1
  },
  address: {
    city: "Douala",
    quartier: "Akwa",
    details: "Near Total Station"
  },
  createdAt: Timestamp,
  updatedAt: Timestamp
}
```

products Collection:

javascript

```
{
  productId: "auto-generated-id",
  name: "iPhone 13 Pro 256GB",
  description: "Original iPhone 13 Pro in excellent condition...",
  price: 450000, // XAF
  category: "smartphones",
  brand: "Apple",
  specifications: {
    storage: "256GB",
    ram: "6GB",
    camera: "12MP Triple Camera",
    battery: "3095mAh",
    screen: "6.1 inch Super Retina XDR",
    processor: "A15 Bionic",
    warranty: "6 months seller warranty"
  },
  imageUrls: [
    "https://res.cloudinary.com/electrohub/image/upload/v1/.../img1.jpg",
    "https://res.cloudinary.com/electrohub/image/upload/v1/.../img2.jpg"
  ],
  videoUrl: "https://res.cloudinary.com/electrohub/video/upload/v1/.../demo.mp4",
  sellerId: "user-id-123",
  status: "active", // active|sold|paused|deleted
  condition: "used-excellent",
  views: 245,
  favorites: 12,
  createdAt: Timestamp,
  updatedAt: Timestamp
}
```

orders Collection:

```
javascript
```

```
{
  orderId: "ORD-20260209-ABC123",
  buyerId: "buyer-user-id",
  sellerId: "seller-user-id",
  productId: "product-id",
  productSnapshot: {
    // Complete product data at time of order
    // Preserved even if seller later edits/deletes listing
    name: "iPhone 13 Pro 256GB",
    price: 450000,
    specifications: {...},
    imageUrls: [...]
  },
  quantity: 1,
  totalAmount: 450000,
  commission: 22500, // 5% platform fee
  sellerReceives: 427500,

  status: "completed", // pending_payment|paid|shipped|delivered|completed|disputed|cancelled
  escrowStatus: "released", // none|held|released|refunded

  paymentMethod: "mtn_mobile_money",
  paymentReference: "MTN-TXN-987654",

  deliveryOTP: "847293",
  otpUsed: true,
  deliveryMethod: "pickup_point",
  deliveryAddress: {
    type: "pickup_point",
    pointName: "Pharmacie du Rond-Point",
    address: "Akwa, Douala",
    contactPhone: "+237699887766"
  },

  timestamps: {
    createdAt: Timestamp("2026-02-09 10:30:00"),
    paidAt: Timestamp("2026-02-09 10:35:00"),
    shippedAt: Timestamp("2026-02-10 14:20:00"),
    deliveredAt: Timestamp("2026-02-11 16:45:00"),
    completedAt: Timestamp("2026-02-11 16:50:00")
  },
```

```
  disputeId: null // Reference if dispute filed
}
```

7.1.2 Composite Indexes

Firestore composite indexes required for efficient queries:

```
Collection: products
Fields: status (ASC), category (ASC), createdAt (DESC)
Use case: Browse active products by category, sorted by newest

Collection: products
Fields: status (ASC), price (ASC)
Use case: Filter active products by price range

Collection: orders
Fields: buyerId (ASC), status (ASC), createdAt (DESC)
Use case: Buyer's order history with status filter

Collection: orders
Fields: sellerId (ASC), status (ASC), createdAt (DESC)
Use case: Seller's orders with status filter

Collection: disputes
Fields: status (ASC), createdAt (ASC)
Use case: Admin dashboard - pending disputes queue
```

7.2 API Integration Design

7.2.1 Mobile Money Integration Architecture

MTN Mobile Money API:

```
javascript
```

```
// Payment initiation
async function initiateMTNPayment(orderData) {
  const endpoint = 'https://proxy.momoapi.mtn.com/collection/v1_0/requesttopay';

  // 1. Get OAuth access token
  const authToken = await getMTNAccessToken();

  // 2. Generate unique reference ID
  const referenceId = generateUUID();

  // 3. Request payment
  const response = await fetch(endpoint, {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${authToken}`,
      'X-Reference-Id': referenceId,
      'X-Target-Environment': 'mtnecameroon',
      'Content-Type': 'application/json',
      'Ocp-Apim-Subscription-Key': process.env.MTN_SUBSCRIPTION_KEY
    },
    body: JSON.stringify({
      amount: orderData.totalAmount,
      currency: 'XAF',
      externalId: orderData.orderId,
      payer: {
        partyIdType: 'MSISDN',
        partyId: orderData.buyerPhone.replace('+237', '') // Remove country code
      },
      payerMessage: `Payment for order ${orderData.orderId}`,
      payeeNote: `ElectroHub order payment`
    })
  });

  if (response.status === 202) {
    // Payment request accepted - user will receive STK push
    return {
      success: true,
      referenceId: referenceId,
      message: 'Payment request sent to user phone'
    };
  } else {
    throw new Error('Payment initiation failed');
  }
```

```
}

// Check payment status
async function checkMTNPaymentStatus(referenceId) {
  const endpoint = `https://proxy.momoapi.mtn.com/collection/v1_0/requesttopay/${referenceId}`;
  const authToken = await getMTNAccessToken();

  const response = await fetch(endpoint, {
    headers: {
      'Authorization': `Bearer ${authToken}`,
      'X-Target-Environment': 'mtnecameroon',
      'Ocp-Apim-Subscription-Key': process.env.MTN_SUBSCRIPTION_KEY
    }
  });

  const data = await response.json();

  return {
    status: data.status, // PENDING|SUCCESSFUL|FAILED
    amount: data.amount,
    currency: data.currency,
    financialTransactionId: data.financialTransactionId
  };
}

// Release escrow to seller
async function releasePaymentToSeller(orderData) {
  const endpoint = 'https://proxy.momoapi.mtn.com/disbursement/v1_0/transfer';
  const authToken = await getMTNDisbursementToken();
  const referenceId = generateUUID();

  const response = await fetch(endpoint, {
    method: 'POST',
    headers: {
      'Authorization': `Bearer ${authToken}`,
      'X-Reference-Id': referenceId,
      'X-Target-Environment': 'mtnecameroon',
      'Content-Type': 'application/json',
      'Ocp-Apim-Subscription-Key': process.env.MTN_DISBURSEMENT_KEY
    },
    body: JSON.stringify({
      amount: orderData.sellerReceives,
      currency: 'XAF',
      externalId: `${orderData.orderId}-payout`,
```

```
    payee: {
      partyIdType: 'MSISDN',
      partyId: orderData.sellerPhone.replace('+237', '')
    },
    payerMessage: `Payout for order ${orderData.orderId}`,
    payeeNote: `ElectroHub sale payment`
  })
});

if (response.status === 202) {
  return {
    success: true,
    referenceId: referenceId,
    message: 'Payment released to seller'
  };
} else {
  throw new Error('Disbursement failed');
}
}
```

Orange Money API (similar structure with Orange-specific endpoints)

7.2.2 SMS Gateway Integration (Twilio)

```
javascript
```

```
// Send OTP via SMS
async function sendDeliveryOTP(phoneNumber, otp, orderId) {
  const accountSid = process.env.TWILIO_ACCOUNT_SID;
  const authToken = process.env.TWILIO_AUTH_TOKEN;
  const twilioPhone = process.env.TWILIO_PHONE_NUMBER;

  const client = require('twilio')(accountSid, authToken);

  const message = await client.messages.create({
    body: `Your ElectroHub delivery OTP for order ${orderId} is: ${otp}. Enter this code in the app to confirm receipt and rele
    from: twilioPhone,
    to: phoneNumber
  });

  return {
    success: true,
    messageSid: message.sid,
    status: message.status
  };
}

// Send order notifications
async function sendOrderNotification(phoneNumber, message) {
  // Similar implementation for transactional notifications
}
```

7.2.3 Cloudinary Integration

javascript

```
// Client-side image upload
async function uploadProductImages(files) {
  const cloudName = 'electrohub';
  const uploadPreset = 'product_images'; // Unsigned preset for client uploads

  const uploadPromises = files.map(async (file) => {
    const formData = new FormData();
    formData.append('file', file);
    formData.append('upload_preset', uploadPreset);
    formData.append('folder', 'products');
    formData.append('transformation', 'q_auto,f_auto,w_1200,c_limit');

    const response = await fetch(
      `https://api.cloudinary.com/v1_1/${cloudName}/image/upload`,
      {
        method: 'POST',
        body: formData
      }
    );

    const data = await response.json();
    return data.secure_url; // HTTPS URL for image
  });

  return await Promise.all(uploadPromises);
}

// Video upload with progress tracking
async function uploadVerificationVideo(file, onProgress) {
  const cloudName = 'electrohub';
  const uploadPreset = 'product_videos';

  const formData = new FormData();
  formData.append('file', file);
  formData.append('upload_preset', uploadPreset);
  formData.append('folder', 'verification_videos');
  formData.append('resource_type', 'video');

  return new Promise((resolve, reject) => {
    const xhr = new XMLHttpRequest();

    xhr.upload.addEventListener('progress', (e) => {
      if (e.lengthComputable) {
```

```
      const percentComplete = (e.loaded / e.total) * 100;
      onProgress(percentComplete);
    }
  });

  xhr.addEventListener('load', () => {
    if (xhr.status === 200) {
      const data = JSON.parse(xhr.responseText);
      resolve(data.secure_url);
    } else {
      reject(new Error('Upload failed'));
    }
  });

  xhr.open('POST', `https://api.cloudinary.com/v1_1/${cloudName}/video/upload`);
  xhr.send(formData);
 });
}
```

7.3 Frontend Design Patterns

7.3.1 Component Structure

```
src/
├── index.html
├── css/
│   ├── main.css
│   ├── components/
│   │   ├── header.css
│   │   ├── product-card.css
│   │   ├── modal.css
│   │   └── forms.css
│   └── pages/
│       ├── home.css
│       ├── product-detail.css
│       └── dashboard.css
├── js/
│   ├── app.js (main entry point)
│   ├── config/
│   │   └── firebase.js
│   ├── services/
│   │   ├── auth.js
│   │   ├── products.js
│   │   ├── orders.js
│   │   ├── payments.js
│   │   └── cloudinary.js
│   ├── components/
│   │   ├── ProductCard.js
│   │   ├── Cart.js
│   │   ├── Modal.js
│   │   └── Notification.js
│   ├── pages/
│   │   ├── home.js
│   │   ├── product-detail.js
│   │   ├── checkout.js
│   │   └── seller-dashboard.js
│   └── utils/
│       ├── validation.js
│       ├── formatting.js
│       └── helpers.js
└── assets/
    ├── images/
    └── icons/
```

7.3.2 Module Pattern Example

javascript

```
// services/products.js
import { db } from '../config/firebase.js';
import {
  collection,
  addDoc,
  query,
  where,
  getDocs,
  doc,
  updateDoc,
  serverTimestamp
} from 'firebase/firestore';

export const ProductService = {

  // Create new product
  async createProduct(productData, userId) {
    try {
      const productsRef = collection(db, 'products');

      const product = {
        ...productData,
        sellerId: userId,
        status: 'active',
        views: 0,
        favorites: 0,
        createdAt: serverTimestamp(),
        updatedAt: serverTimestamp()
      };

      const docRef = await addDoc(productsRef, product);

      // Update user to seller if first product
      await this.updateUserToSeller(userId);

      return { success: true, productId: docRef.id };
    } catch (error) {
      console.error('Error creating product:', error);
      throw error;
    }
  },

  // Get products with filters
```

```
async getProducts(filters = {}) {
  try {
    const productsRef = collection(db, 'products');
    let q = query(productsRef, where('status', '==', 'active'));

    if (filters.category) {
      q = query(q, where('category', '==', filters.category));
    }

    if (filters.minPrice) {
      q = query(q, where('price', '>=', filters.minPrice));
    }

    if (filters.maxPrice) {
      q = query(q, where('price', '<=', filters.maxPrice));
    }

    const querySnapshot = await getDocs(q);
    const products = [];

    querySnapshot.forEach((doc) => {
      products.push({ id: doc.id, ...doc.data() });
    });

    return products;
  } catch (error) {
    console.error('Error fetching products:', error);
    throw error;
  }
},

// Update user to seller status
async updateUserToSeller(userId) {
  try {
    const userRef = doc(db, 'users', userId);
    await updateDoc(userRef, {
      isSeller: true,
      updatedAt: serverTimestamp()
    });

    // Initialize seller analytics
    const analyticsRef = collection(db, 'analytics');
    await addDoc(analyticsRef, {
      sellerId: userId,
```

```
        totalSales: 0,
        totalRevenue: 0,
        escrowHeld: 0,
        escrowReleased: 0,
        productListings: 1,
        activeListings: 1,
        lastUpdated: serverTimestamp()
      });
    } catch (error) {
      console.error('Error updating user to seller:', error);
    }
  },

  // Increment product views
  async incrementViews(productId) {
    try {
      const productRef = doc(db, 'products', productId);
      const currentViews = await this.getProduct(productId);

      await updateDoc(productRef, {
        views: (currentViews?.views || 0) + 1
      });
    } catch (error) {
      console.error('Error incrementing views:', error);
    }
  }
};
```

7.3.3 State Management Pattern

javascript

```
// Simple state management for cart
class CartManager {
  constructor() {
    this.cart = this.loadCart();
    this.listeners = [];
  }

  // Load cart from Firestore or localStorage fallback
  async loadCart() {
    const user = auth.currentUser;
    if (user) {
      // Load from Firestore
      const cartRef = doc(db, 'carts', user.uid);
      const cartDoc = await getDoc(cartRef);
      return cartDoc.exists() ? cartDoc.data().items : [];
    } else {
      // Load from localStorage for anonymous users
      const stored = localStorage.getItem('electrohub_cart');
      return stored ? JSON.parse(stored) : [];
    }
  }

  // Add item to cart
  async addItem(product, quantity = 1) {
    const existingItem = this.cart.find(item => item.productId === product.id);

    if (existingItem) {
      existingItem.quantity += quantity;
    } else {
      this.cart.push({
        productId: product.id,
        name: product.name,
        price: product.price,
        imageUrl: product.imageUrls[0],
        quantity: quantity,
        addedAt: new Date().toISOString()
      });
    }

    await this.saveCart();
    this.notifyListeners();
  }
```

```
// Remove item from cart
async removeItem(productId) {
  this.cart = this.cart.filter(item => item.productId !== productId);
  await this.saveCart();
  this.notifyListeners();
}

// Update quantity
async updateQuantity(productId, quantity) {
  const item = this.cart.find(item => item.productId === productId);
  if (item) {
    item.quantity = quantity;
    await this.saveCart();
    this.notifyListeners();
  }
}

// Save cart to Firestore or localStorage
async saveCart() {
  const user = auth.currentUser;
  if (user) {
    const cartRef = doc(db, 'carts', user.uid);
    await setDoc(cartRef, { items: this.cart, updatedAt: serverTimestamp() });
  } else {
    localStorage.setItem('electrohub_cart', JSON.stringify(this.cart));
  }
}

// Get total price
getTotalPrice() {
  return this.cart.reduce((total, item) => total + (item.price * item.quantity), 0);
}

// Subscribe to cart changes
subscribe(callback) {
  this.listeners.push(callback);
}

// Notify all listeners
notifyListeners() {
  this.listeners.forEach(callback => callback(this.cart));
}

// Clear cart
```

```
  async clear() {
    this.cart = [];
    await this.saveCart();
    this.notifyListeners();
  }
}

// Export singleton instance
export const cartManager = new CartManager();
```

---

8. Implementation Details

8.1 Key Features Implementation

8.1.1 Product Listing with Video Verification

HTML Structure:

```
html
```

```
<!-- Add Product Form -->
<form id="addProductForm" class="product-form">
  <div class="form-group">
    <label for="productName">Product Name *</label>
    <input type="text" id="productName" required maxlength="100">
  </div>

  <div class="form-group">
    <label for="category">Category *</label>
    <select id="category" required>
      <option value="">Select category</option>
      <option value="smartphones">Smartphones</option>
      <option value="laptops">Laptops & Computers</option>
      <option value="tablets">Tablets</option>
      <option value="accessories">Accessories</option>
      <option value="audio">Audio Equipment</option>
    </select>
  </div>

  <div class="form-group">
    <label for="price">Price (XAF) *</label>
    <input type="number" id="price" required min="1000" max="10000000">
  </div>

  <div class="form-group">
    <label for="description">Description *</label>
    <textarea id="description" required rows="6" maxlength="2000"></textarea>
  </div>

  <div class="form-group">
    <label>Specifications</label>
    <div id="specificationsContainer">
      <input type="text" placeholder="Brand" name="brand">
      <input type="text" placeholder="Model" name="model">
      <input type="text" placeholder="Storage" name="storage">
      <input type="text" placeholder="RAM" name="ram">
      <!-- Dynamic fields based on category -->
    </div>
  </div>

  <div class="form-group">
    <label for="images">Product Images * (Max 10, 5MB each)</label>
    <div class="image-upload-area" id="imageUploadArea">
```

```
    <input type="file" id="images" accept="image/*" multiple required>
    <div class="upload-prompt">
      <i class="upload-icon"></i>
      <p>Click or drag images here</p>
    </div>
  </div>
  <div id="imagePreviewContainer" class="image-preview-grid"></div>
 </div>

 <div class="form-group">
  <label for="video">Verification Video (Optional, max 90s, 50MB)</label>
  <input type="file" id="video" accept="video/mp4,video/mov">
  <p class="help-text">Record a short video showing the product working and demonstrating specifications</p>
  <div id="videoPreview"></div>
  <div id="uploadProgress" class="progress-bar" style="display:none;">
    <div class="progress-fill"></div>
    <span class="progress-text">0%</span>
  </div>
 </div>

 <button type="submit" class="btn-primary">Publish Product</button>
</form>
```

JavaScript Implementation:

```
javascript
```

```
// Handle product form submission
document.getElementById('addProductForm').addEventListener('submit', async (e) => {
  e.preventDefault();

  try {
    // Show loading state
    showLoading('Uploading product...');

    // 1. Collect form data
    const formData = {
      name: document.getElementById('productName').value,
      category: document.getElementById('category').value,
      price: parseInt(document.getElementById('price').value),
      description: document.getElementById('description').value,
      specifications: collectSpecifications()
    };

    // 2. Upload images to Cloudinary
    const imageFiles = document.getElementById('images').files;
    const imageUrls = await uploadProductImages(Array.from(imageFiles));
    formData.imageUrls = imageUrls;

    // 3. Upload video if provided
    const videoFile = document.getElementById('video').files[0];
    if (videoFile) {
      formData.videoUrl = await uploadVerificationVideo(videoFile, (progress) => {
        updateUploadProgress(progress);
      });
    }

    // 4. Create product in Firestore
    const user = auth.currentUser;
    const result = await ProductService.createProduct(formData, user.uid);

    // 5. Show success and redirect
    hideLoading();
    showNotification('Product published successfully!', 'success');
    setTimeout(() => {
      window.location.href = '/seller-dashboard.html';
    }, 1500);

  } catch (error) {
    hideLoading();
```

```
    showNotification('Error publishing product: ' + error.message, 'error');
    console.error(error);
  }
});

// Collect specifications from dynamic inputs
function collectSpecifications() {
  const specs = {};
  const specInputs = document.querySelectorAll('#specificationsContainer input');

  specInputs.forEach(input => {
    if (input.value.trim()) {
      specs[input.name] = input.value.trim();
    }
  });

  return specs;
}

// Image preview functionality
document.getElementById('images').addEventListener('change', (e) => {
  const files = Array.from(e.target.files);
  const previewContainer = document.getElementById('imagePreviewContainer');
  previewContainer.innerHTML = '';

  files.forEach((file, index) => {
    const reader = new FileReader();

    reader.onload = (event) => {
      const preview = document.createElement('div');
      preview.className = 'image-preview-item';
      preview.innerHTML = `
        <img src="${event.target.result}" alt="Preview ${index + 1}">
        <button type="button" class="remove-image" data-index="${index}">×</button>
        ${index === 0 ? '<span class="primary-badge">Primary</span>' : ''}
      `;
      previewContainer.appendChild(preview);
    };

    reader.readAsDataURL(file);
  });
});

// Update upload progress
```

```
function updateUploadProgress(percent) {
  const progressBar = document.getElementById('uploadProgress');
  const fill = progressBar.querySelector('.progress-fill');
  const text = progressBar.querySelector('.progress-text');

  progressBar.style.display = 'block';
  fill.style.width = percent + '%';
  text.textContent = Math.round(percent) + '%';
}
```

8.1.2 Escrow Payment Implementation

javascript

```
// Checkout and payment service
export const CheckoutService = {

  // Initiate checkout process
  async initiateCheckout(cartItems, deliveryDetails) {
    try {
      const user = auth.currentUser;
      if (!user) throw new Error('User must be logged in');

      // Calculate totals
      const subtotal = cartItems.reduce((sum, item) => sum + (item.price * item.quantity), 0);
      const commission = Math.round(subtotal * 0.05); // 5% platform fee
      const total = subtotal;

      // Create order documents for each item
      const orderPromises = cartItems.map(async (item) => {
        // Get full product details
        const productDoc = await getDoc(doc(db, 'products', item.productId));
        const productData = productDoc.data();

        // Create order
        const orderRef = collection(db, 'orders');
        const orderId = `ORD-${Date.now()}-${generateShortId()}`;

        const order = {
          orderId: orderId,
          buyerId: user.uid,
          sellerId: productData.sellerId,
          productId: item.productId,
          productSnapshot: {
            name: productData.name,
            price: productData.price,
            specifications: productData.specifications,
            imageUrls: productData.imageUrls
          },
          quantity: item.quantity,
          totalAmount: item.price * item.quantity,
          commission: Math.round((item.price * item.quantity) * 0.05),
          sellerReceives: Math.round((item.price * item.quantity) * 0.95),

          status: 'pending_payment',
          escrowStatus: 'none',
```

```
        paymentMethod: deliveryDetails.paymentMethod,

        deliveryOTP: generateOTP(),
        otpUsed: false,
        deliveryMethod: deliveryDetails.method,
        deliveryAddress: deliveryDetails.address,

        timestamps: {
          createdAt: serverTimestamp()
        }
      };

      await addDoc(orderRef, order);
      return order;
    });

    const orders = await Promise.all(orderPromises);

    // Initiate payment
    const paymentResult = await this.processPayment(
      total,
      user.phoneNumber,
      orders[0].orderId, // Primary order ID for reference
      deliveryDetails.paymentMethod
    );

    return {
      success: true,
      orders: orders,
      paymentReference: paymentResult.referenceId
    };

  } catch (error) {
    console.error('Checkout error:', error);
    throw error;
  }
},

// Process mobile money payment
async processPayment(amount, phoneNumber, orderId, method) {
  try {
    let result;

    if (method === 'mtn_mobile_money') {
```

```
      result = await this.processMTNPayment(amount, phoneNumber, orderId);
    } else if (method === 'orange_money') {
      result = await this.processOrangePayment(amount, phoneNumber, orderId);
    } else {
      throw new Error('Invalid payment method');
    }

    return result;

  } catch (error) {
    console.error('Payment processing error:', error);
    throw error;
  }
},

// MTN Mobile Money payment
async processMTNPayment(amount, phoneNumber, orderId) {
  // Call MTN API (server-side function)
  const response = await fetch('/api/payment/mtn/initiate', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      amount: amount,
      phoneNumber: phoneNumber,
      orderId: orderId
    })
  });

  if (!response.ok) throw new Error('Payment initiation failed');

  const data = await response.json();

  // Poll for payment confirmation
  const confirmed = await this.pollPaymentStatus(data.referenceId, 'mtn');

  if (confirmed) {
    // Update order status
    await this.updateOrderAfterPayment(orderId, data.referenceId);
  }

  return data;
},

// Poll payment status
```

```
async pollPaymentStatus(referenceId, provider, maxAttempts = 30) {
  for (let i = 0; i < maxAttempts; i++) {
    await new Promise(resolve => setTimeout(resolve, 2000)); // Wait 2s

    const response = await fetch(`/api/payment/${provider}/status/${referenceId}`);
    const data = await response.json();

    if (data.status === 'SUCCESSFUL') {
      return true;
    } else if (data.status === 'FAILED') {
      throw new Error('Payment failed');
    }
    // Continue polling if PENDING
  }

  throw new Error('Payment timeout');
},

// Update order after successful payment
async updateOrderAfterPayment(orderId, paymentReference) {
  const orderQuery = query(
    collection(db, 'orders'),
    where('orderId', '==', orderId)
  );

  const snapshot = await getDocs(orderQuery);

  snapshot.forEach(async (orderDoc) => {
    await updateDoc(orderDoc.ref, {
      status: 'paid',
      escrowStatus: 'held',
      paymentReference: paymentReference,
      'timestamps.paidAt': serverTimestamp()
    });

    // Send OTP to buyer
    const orderData = orderDoc.data();
    await this.sendDeliveryOTP(orderData);

    // Notify seller
    await this.notifySeller(orderData);
  });
},
```

```
// Send delivery OTP via SMS
async sendDeliveryOTP(orderData) {
  await fetch('/api/sms/send-otp', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      phoneNumber: orderData.buyerPhone,
      otp: orderData.deliveryOTP,
      orderId: orderData.orderId
    })
  });
},

// Confirm delivery with OTP
async confirmDelivery(orderId, otp) {
  try {
    // Find order
    const orderQuery = query(
      collection(db, 'orders'),
      where('orderId', '==', orderId)
    );

    const snapshot = await getDocs(orderQuery);

    if (snapshot.empty) {
      throw new Error('Order not found');
    }

    const orderDoc = snapshot.docs[0];
    const orderData = orderDoc.data();

    // Validate OTP
    if (orderData.deliveryOTP !== otp) {
      throw new Error('Invalid OTP');
    }

    if (orderData.otpUsed) {
      throw new Error('OTP already used');
    }

    // Check if OTP expired (48 hours)
    const paidAt = orderData.timestamps.paidAt.toDate();
    const now = new Date();
    const hoursSincePaid = (now - paidAt) / (1000 * 60 * 60);
```

```
    if (hoursSincePaid > 48) {
      throw new Error('OTP expired');
    }

    // Release payment to seller
    await this.releaseEscrowPayment(orderData);

    // Update order
    await updateDoc(orderDoc.ref, {
      status: 'completed',
      escrowStatus: 'released',
      otpUsed: true,
      'timestamps.deliveredAt': serverTimestamp(),
      'timestamps.completedAt': serverTimestamp()
    });

    // Update seller analytics
    await this.updateSellerAnalytics(orderData);

    // Send notifications
    await this.notifyOrderCompletion(orderData);

    return { success: true, message: 'Delivery confirmed, payment released' };

  } catch (error) {
    console.error('Delivery confirmation error:', error);
    throw error;
  }
},

// Release escrow payment to seller
async releaseEscrowPayment(orderData) {
  await fetch('/api/payment/release', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      orderId: orderData.orderId,
      sellerId: orderData.sellerId,
      amount: orderData.sellerReceives,
      paymentMethod: orderData.paymentMethod
    })
  });
}
```

```
};

// Generate 6-digit OTP
function generateOTP() {
  return Math.floor(100000 + Math.random() * 900000).toString();
}

// Generate short ID
function generateShortId() {
  return Math.random().toString(36).substring(2, 8).toUpperCase();
}
```

8.1.3 Seller Dashboard Implementation

javascript

```
// Seller Dashboard Controller
class SellerDashboard {
  constructor() {
    this.sellerId = null;
    this.analytics = null;
    this.init();
  }

  async init() {
    const user = auth.currentUser;
    if (!user) {
      window.location.href = '/login.html';
      return;
    }

    this.sellerId = user.uid;
    await this.loadDashboardData();
    this.setupEventListeners();
  }

  async loadDashboardData() {
    try {
      // Load seller analytics
      const analyticsQuery = query(
        collection(db, 'analytics'),
        where('sellerId', '==', this.sellerId)
      );

      const analyticsSnapshot = await getDocs(analyticsQuery);
      if (!analyticsSnapshot.empty) {
        this.analytics = analyticsSnapshot.docs[0].data();
      }

      // Load products
      await this.loadProducts();

      // Load recent orders
      await this.loadOrders();

      // Render overview
      this.renderOverview();

    } catch (error) {
```

```
    console.error('Error loading dashboard:', error);
  }
}

renderOverview() {
  const overview = document.getElementById('dashboardOverview');

  overview.innerHTML = `
    <div class="stats-grid">
      <div class="stat-card">
        <div class="stat-icon">📦</div>
        <div class="stat-content">
          <h3>${this.analytics?.activeListings || 0}</h3>
          <p>Active Listings</p>
        </div>
      </div>

      <div class="stat-card">
        <div class="stat-icon">💰</div>
        <div class="stat-content">
          <h3>${formatCurrency(this.analytics?.totalRevenue || 0)}</h3>
          <p>Total Revenue</p>
        </div>
      </div>

      <div class="stat-card">
        <div class="stat-icon">⏳</div>
        <div class="stat-content">
          <h3>${formatCurrency(this.analytics?.escrowHeld || 0)}</h3>
          <p>In Escrow</p>
        </div>
      </div>

      <div class="stat-card">
        <div class="stat-icon">✅</div>
        <div class="stat-content">
          <h3>${this.analytics?.totalSales || 0}</h3>
          <p>Completed Sales</p>
        </div>
      </div>
    </div>

    <div class="revenue-chart">
      <h3>Revenue Over Time</h3>
```

```
      <canvas id="revenueChart"></canvas>
    </div>
  `;

  // Render chart
  this.renderRevenueChart();
}

async loadProducts() {
  const productsQuery = query(
    collection(db, 'products'),
    where('sellerId', '==', this.sellerId),
    where('status', '!=', 'deleted')
  );

  const snapshot = await getDocs(productsQuery);
  const products = [];

  snapshot.forEach(doc => {
    products.push({ id: doc.id, ...doc.data() });
  });

  this.renderProducts(products);
}

renderProducts(products) {
  const container = document.getElementById('productsContainer');

  if (products.length === 0) {
    container.innerHTML = '<p class="empty-state">No products yet. Start by adding your first product!</p>';
    return;
  }

  container.innerHTML = `
    <table class="products-table">
      <thead>
        <tr>
          <th>Product</th>
          <th>Price</th>
          <th>Status</th>
          <th>Views</th>
          <th>Actions</th>
        </tr>
      </thead>
```

```
      <tbody>
        ${products.map(product => `
          <tr>
            <td>
              <div class="product-cell">
                <img src="${product.imageUrls[0]}" alt="${product.name}">
                <span>${product.name}</span>
              </div>
            </td>
            <td>${formatCurrency(product.price)}</td>
            <td><span class="status-badge status-${product.status}">${product.status}</span></td>
            <td>${product.views}</td>
            <td>
              <div class="action-buttons">
                <button class="btn-icon" onclick="dashboard.editProduct('${product.id}')" title="Edit">
                  <i class="icon-edit"></i>
                </button>
                <button class="btn-icon" onclick="dashboard.toggleProductStatus('${product.id}')" title="Pause/Activate">
                  <i class="icon-pause"></i>
                </button>
                <button class="btn-icon btn-danger" onclick="dashboard.deleteProduct('${product.id}')" title="Delete">
                  <i class="icon-delete"></i>
                </button>
              </div>
            </td>
          </tr>
        `).join('')}
      </tbody>
    </table>
  `;
}

async loadOrders() {
  const ordersQuery = query(
    collection(db, 'orders'),
    where('sellerId', '==', this.sellerId),
    orderBy('timestamps.createdAt', 'desc'),
    limit(20)
  );

  const snapshot = await getDocs(ordersQuery);
  const orders = [];

  snapshot.forEach(doc => {
```

```
    orders.push({ id: doc.id, ...doc.data() });
  });

  this.renderOrders(orders);
}

renderOrders(orders) {
  const container = document.getElementById('ordersContainer');

  if (orders.length === 0) {
    container.innerHTML = '<p class="empty-state">No orders yet</p>';
    return;
  }

  container.innerHTML = `
    <table class="orders-table">
      <thead>
        <tr>
          <th>Order ID</th>
          <th>Product</th>
          <th>Amount</th>
          <th>Status</th>
          <th>Date</th>
          <th>Actions</th>
        </tr>
      </thead>
      <tbody>
        ${orders.map(order => `
          <tr>
            <td><code>${order.orderId}</code></td>
            <td>${order.productSnapshot.name}</td>
            <td>${formatCurrency(order.sellerReceives)}</td>
            <td>
              <span class="status-badge status-${order.status}">${order.status}</span>
              <br>
              <small>Escrow: ${order.escrowStatus}</small>
            </td>
            <td>${formatDate(order.timestamps.createdAt)}</td>
            <td>
              ${order.status === 'paid' ? `
                <button class="btn-sm btn-primary" onclick="dashboard.markAsShipped('${order.id}')">
                  Mark Shipped
                </button>
              ` : ''}
```

```
            <button class="btn-sm" onclick="dashboard.viewOrderDetails('${order.id}')">
              View Details
            </button>
          </td>
        </tr>
      `).join('')}
    </tbody>
  </table>
`;
}

async markAsShipped(orderId) {
  try {
    const confirmed = confirm('Have you shipped this product?');
    if (!confirmed) return;

    const orderRef = doc(db, 'orders', orderId);
    await updateDoc(orderRef, {
      status: 'shipped',
      'timestamps.shippedAt': serverTimestamp()
    });

    // Notify buyer
    const orderDoc = await getDoc(orderRef);
    const orderData = orderDoc.data();

    await fetch('/api/notifications/order-shipped', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        orderId: orderData.orderId,
        buyerId: orderData.buyerId
      })
    });

    showNotification('Order marked as shipped. Buyer notified.', 'success');
    await this.loadOrders();

  } catch (error) {
    showNotification('Error updating order: ' + error.message, 'error');
  }
}
}
```

```
// Utility functions
function formatCurrency(amount) {
  return new Intl.NumberFormat
```